



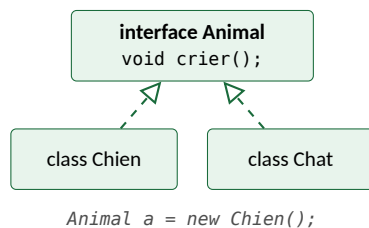
Des interfaces aux Streams API

Programmation orientée objet et fonctionnelle en Java — *fiche de synthèse*

L2 Informatique · prérequis : classes, objets, héritage, ArrayList · Java 8+

1. L'interface : un contrat

Une interface déclare *ce qu'un objet sait faire, jamais comment*. Toute classe qui l'implémente s'engage à fournir les méthodes déclarées.



Le type *déclaré* est l'interface, le type *réel* est la classe : c'est le polymorphisme. Une méthode qui reçoit un `Animal` accepte toute implémentation.

Interface ou classe abstraite ?

	Interface	Classe abstraite
Rôle	un contrat	une base commune
Héritage	plusieurs possibles	une seule
Objectif	polymorphisme	factorisation de code

2. Interface fonctionnelle

Interface possédant **une seule méthode abstraite**. C'est la condition pour qu'une lambda puisse la cibler : sans ambiguïté, le compilateur sait à quelle signature rattacher l'expression.

L'annotation `@FunctionalInterface` est facultative mais recommandée : elle fait vérifier cette contrainte par le compilateur.

3. Expression lambda

Implémentation courte et anonyme d'une interface fonctionnelle. Remplace la classe anonyme.



Cas	Écriture
aucun paramètre	() -> ... parenthèses obligatoires
un seul paramètre	x -> ... parenthèses facultatives
plusieurs paramètres	(a, b) -> ... parenthèses obligatoires
corps sur plusieurs lignes	accolades + return

4. Interfaces de java.util.function

Interface	Méthode	Entrée → sortie
<code>Consumer<T></code>	<code>accept</code>	<code>T → rien</code>
<code>Supplier<T></code>	<code>get</code>	<code>rien → T</code>
<code>Predicate<T></code>	<code>test</code>	<code>T → boolean</code>
<code>Function<T,R></code>	<code>apply</code>	<code>T → R</code>
<code>UnaryOperator<T></code>	<code>apply</code>	<code>T → T</code>
<code>BinaryOperator<T></code>	<code>apply</code>	<code>T, T → T</code>

5. Référence de méthode

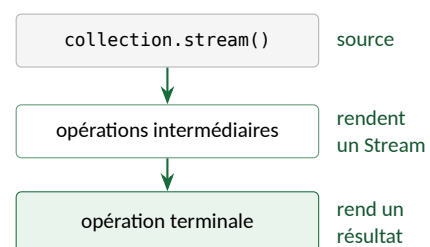
Écriture abrégée d'une lambda qui se contente d'appeler une méthode existante avec les mêmes arguments.

Forme	Exemple → lambda équivalente
méthode statique	<code>Math::abs → x -> Math.abs(x)</code>
objet précis	<code>System.out::println</code>
méthode d'un type	<code>String::toUpperCase</code>
constructeur	<code>ArrayList::new</code>

Dès qu'un calcul s'ajoute à l'appel, revenir à la lambda.

6. Le pipeline Stream

Un `Stream` **n'est pas une collection** : il ne stocke rien, il décrit un traitement.



Trois règles absolues

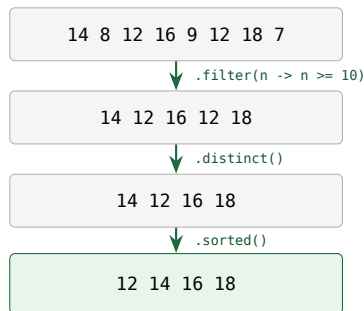
1. Sans opération terminale, **rien ne s'exécute** — et aucune erreur n'apparaît.
2. Un `Stream` ne se parcourt **qu'une seule fois** ; le réutiliser lève `IllegalStateException`.
3. Un `Stream` **ne modifie pas** la collection d'origine.

L'évaluation est *paresseuse* : chaque élément traverse le pipeline en entier avant que le suivant n'y entre. Ce n'est pas un traitement par étages successifs.

7. Opérations intermédiaires

Opération	Effet
filter	conserve les éléments vérifiant une condition
map	transforme chaque élément
flatMap	aplatit des collections imbriquées
sorted	trie (comparateur optionnel)
distinct	supprime les doublons
limit	garde les n premiers
skip	ignore les n premiers
peek	observe sans transformer (débugage)

Exemple d'enchaînement, valeurs réelles :



8. Opérations terminales

Opération	Résultat
forEach	traite chaque élément, ne rend rien
count	un long
min/max	un Optional
reduce	une valeur unique agrégée
collect	une collection ou un résultat structuré

L'Optional

min et max rendent un Optional car le Stream peut être vide. Afficher directement donne Optional[18]. Déballer avec ifPresent(...) ou orElse(...); get() sur un Optional vide lève NoSuchElementException.

9. Collectors

Collecteur	Produit
toList / toSet	une List / un Set
joining(", ")	une chaîne concaténée
counting()	un effectif
averagingDouble	une moyenne
groupingBy	une Map clé → liste
partitioningBy	une Map à deux clés true/false
summarizingDouble	min, max, moyenne, somme, effectif

groupingBy rend une HashMap : l'ordre des clés n'est pas garanti. partitioningBy rend toujours exactement deux clés, même si l'un des groupes est vide.

10. Pièges fréquents

Symptôme	Cause
rien ne s'affiche	pipeline sans opération terminale
IllegalStateException	Stream déjà consommé
Optional[16.0] affiché	Optional non déballé
ordre de groupes inattendu	groupingBy rend une HashMap
liste inchangée	un Stream ne modifie pas la source

11. Vocabulaire

Terme	Définition
Contrat	ce qu'une classe s'engage à savoir faire
Méthode abstraite	méthode déclarée sans corps
Pipeline	source → intermédiaires → terminale
Paresse	rien ne s'exécute avant la terminale
Consommation	un Stream ne se parcourt qu'une fois
Déclaratif	décrire le résultat, pas les étapes

Pour aller plus loin

Comparator.comparing(...) et .reversed() pour les tris; IntStream et DoubleStream pour éviter l'autoboxing; groupingBy(critere, counting()) pour combiner deux collecteurs; parallelStream() sur de très grands volumes seulement.